

G-CAF

GovDigital Composable Architecture Framework

Volume 2

Framework Core — Tiers, Frames, Principles & Classifications

v2.0

Table of Contents

Chapter 1 3-Tier Architecture Overview

How tiers stack and interact

Chapter 2 Experience Tier

Where users interact with government services

Chapter 3 Capability Tier

What we deliver — PBCs and Domains

Chapter 4 Platform Tier

Integration, Developer, Infrastructure sub-tiers

Chapter 5 Frames Overview

Why cross-cutting concerns wrap, not stack

Chapter 6 Governance Frame

Design-time policies that shape what we build

Chapter 7 Operations Frame

Runtime policies that ensure systems run

Chapter 8 Architecture Principles

17 principles governing all decisions

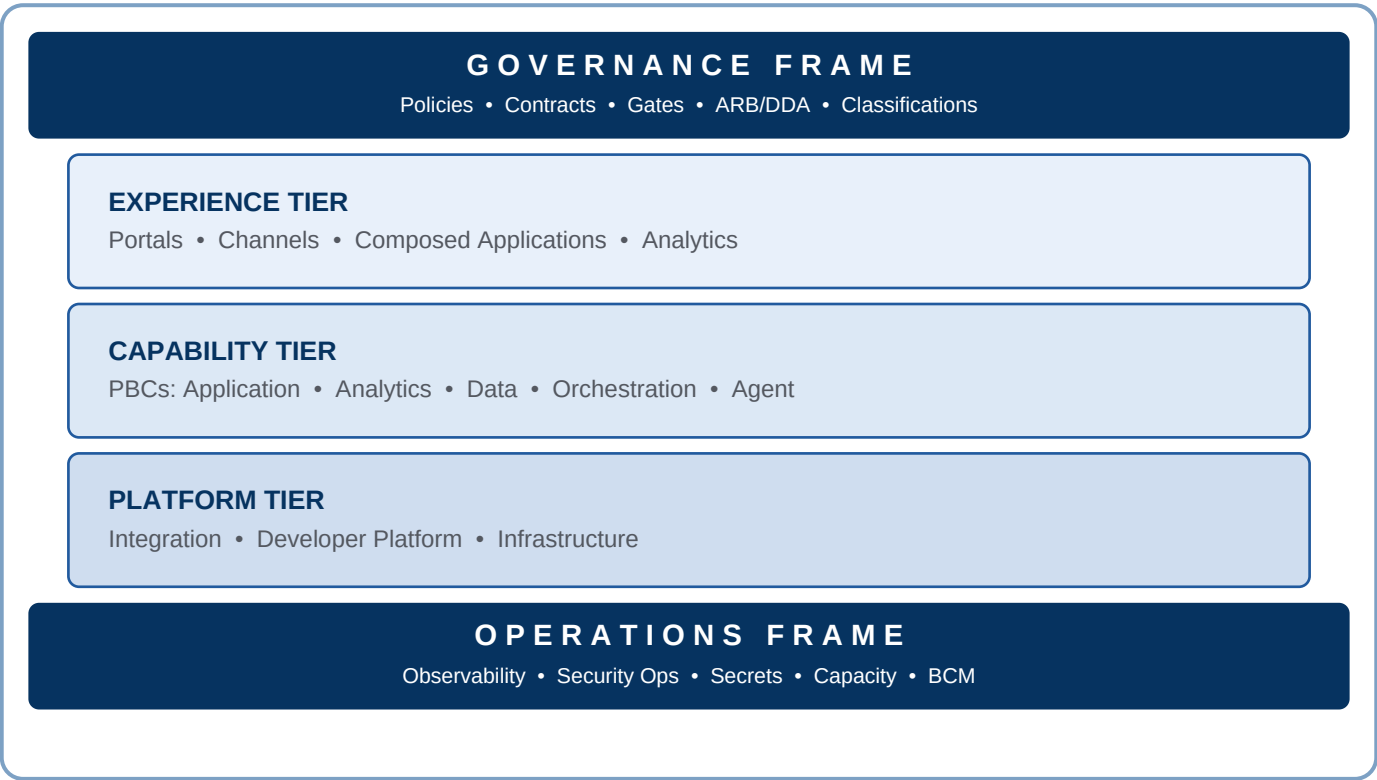
Chapter 9 Classifications

PBC types, criticality, sourcing, stability, maturity

Chapter 1

3-Tier Architecture Overview

G-CAF organizes composable government architecture into three vertical tiers. Tiers stack — Experiences compose Capabilities, which run on Platforms.



Tier Summary

Tier	Purpose	Key Question	Change Rate
Experience	User interaction	How do users access services?	High
Capability	Business building blocks	What do we deliver?	Medium
Platform	Shared infrastructure	What do teams build on?	Low

Dependency Flow

Higher tiers depend on lower tiers: Experience → Capability → Platform. Lower tiers do not depend on higher tiers. Contracts mediate all cross-tier communication. Frames (Governance, Operations) apply to all tiers.

Stability Gradient

Tier	Stability	Change Frequency	Examples
Experience	Volatile	Weekly/Monthly	Portal redesign, new channel
Capability	Moderate	Monthly/Quarterly	New PBC, feature addition
Platform	Stable	Quarterly/Yearly	Infrastructure upgrade

Changes should flow downward carefully. A platform change affects all capabilities and experiences above it.

Chapter 2

Experience Tier

The Experience Tier defines **where users interact with government services**. Experiences are compositions — they assemble capabilities from the Capability Tier into user-facing services.

Core Principles

- 1. Experiences Are Compositions, Not Monoliths** — An experience assembles capabilities; it does not contain them. Business logic belongs in PBCs, not in the UI layer.
- 2. Channel-Agnostic Capability Design** — Capabilities work across any channel (Web, Mobile, API, Chatbot). The experience adapts, not the capability.
- 3. Role-Based Assembly** — Experiences are assembled based on user roles and journeys.

Portals

Portal	Audience	Purpose
G2C	Citizens	Public-facing government services
G2B	Businesses	Registration, permits, compliance
G2G	Government	Inter-agency collaboration and data exchange
G2E	Employees	Internal workforce services

Channels

Channel	Description	Use Case
Web	Browser-based responsive	Primary desktop/laptop access
Mobile	Native or PWA	On-the-go access
Kiosk	Self-service terminals	Public locations
API	Headless/programmatic	Integration, automation
Voice	IVR, smart speakers	Accessibility, convenience
Chatbot	Conversational AI	24/7 support, simple queries

Composition Patterns

Micro-Frontend Composition — A portal shell assembles micro-frontends from different PBCs. Each PBC owns its UI fragment, deployed independently.

API-First Composition — Mobile apps and lightweight clients use a Backend-for-Frontend (BFF) layer that aggregates PBC APIs into experience-specific responses.

Governance Requirements

Requirement	Description
Accessibility	WCAG 2.1 AA compliance for all public-facing experiences
Localization	Arabic/English support, RTL layouts
Performance	Core Web Vitals compliance
Analytics	Standardized event tracking
Security	SSO integration, session management

Anti-Patterns

Anti-Pattern	Problem	Resolution
Monolithic Portal	Business logic embedded in UI	Extract logic to PBCs
Channel-Specific APIs	Different APIs per channel	Channel-agnostic PBC APIs
Copy-Paste Composition	Duplicated UI code	Shared component library
Experience-Level Data	Data stored in UI layer	Data belongs in PBCs

Chapter 3

Capability Tier

The Capability Tier defines **what we deliver** — the business building blocks. This tier contains Packaged Business Capabilities (PBCs) organized within Domains.

Core Principles

- 1. PBC is the Unit of Governance** — Every capability is owned, classified, contracted, and governed as a PBC.
- 2. Domains Define Ownership Boundaries** — Bounded contexts that define where ownership and language apply.
- 3. Contracts Enable Independence** — PBCs evolve independently because contracts define stable interfaces.

Six PBC Types

Type	Primary Function	Data Relationship	Examples
Application PBC	Business entity lifecycle	Owns data (Source of Truth)	License, Payment, Citizen
Analytics PBC	ML/AI predictions & insights	Consumes from Data Fabric	Fraud Detection, Scoring
Data PBC	Reference data & search	Operates on Data Fabric	Address Validation, Lookup
Orchestration PBC	Multi-PBC coordination	Process state only	Multi-Agency Approval
Digital Twin PBC	Physical asset model	Owns asset & time-series	Smart Meter, Fleet Vehicle
Agent PBC	Autonomous AI with guardrails	Tools, memory, guardrails	Document Classifier

Data Fabric Relationships

PBC Type	Data Fabric Role
Application PBC	Feeds data to Data Fabric (source of truth)
Analytics PBC	Consumes from Data Fabric (no ownership)
Data PBC	Operates on Data Fabric (reference data, search)
Orchestration PBC	No direct Data Fabric relationship (process state only)
Digital Twin PBC	Feeds asset data to Data Fabric

Domain Model Components

Bounded Contexts — Semantic boundaries within which a domain model applies. Each context has its own language and ownership.

Aggregates — Clusters of domain objects treated as a unit with invariants (e.g., License Aggregate: Applicant, Fee, Document).

Domain Events — Business events that cross boundaries and trigger downstream actions (e.g., LicenseIssued → Notification PBC).

Ubiquitous Language — Shared vocabulary within a domain that appears in code, documentation, and conversation.

Selecting PBC Type

If the PBC...	Then Type is...
Manages a business entity (create, update, delete)	Application PBC
Provides ML/AI predictions or insights	Analytics PBC
Provides lookups, reference data, or search	Data PBC
Coordinates workflows across multiple PBCs	Orchestration PBC
Represents a physical asset digitally	Digital Twin PBC
Operates autonomously with AI reasoning	Agent PBC

Anti-Patterns

Anti-Pattern	Problem	Resolution
God PBC	One PBC does everything	Split by domain
Orphan PBC	No clear owner	Assign ownership or retire
Coupled PBCs	Direct database sharing	Contract-based integration
Missing Contract	No formal interface	Define and version contract
Domain Leakage	Language crosses boundaries	Enforce bounded contexts

Chapter 4

Platform Tier

The Platform Tier defines **what teams build on** — shared infrastructure and services. Three sub-tiers: Integration, Developer, and Infrastructure.

Core Principles

- 1. Platforms Enable, Not Block** — Self-service capabilities that accelerate delivery.
- 2. Stability Increases Toward Infrastructure** — Lower sub-tiers change less frequently.
- 3. Contract-Based Integration** — All platform services accessed through versioned contracts.

Integration Sub-Tier

Component	Purpose
API Gateway	Centralized API management, security, rate limiting
Event Bus	Asynchronous messaging, event streaming, replay
Schema Registry	Schema versioning and compatibility checking
Data Fabric	Unified data access, catalog, lineage, quality
Orchestration	Workflow coordination, saga patterns

Developer Sub-Tier

Component	Purpose
Internal Developer Platform (IDP)	Self-service platform for developers
Developer Portal	Documentation, onboarding, support
Service Catalog	Discoverable services and PBCs
Golden Paths	Opinionated templates for common patterns
CI/CD Pipelines	Automated build, test, deploy
Composition Platform	UI composition tools
Tooling Standards	Approved tools for data, analytics, ML

Infrastructure Sub-Tier

Component	Purpose	Stability
Digital Public Infrastructure (DPI)	National-scale shared infrastructure	Very High

Component	Purpose	Stability
Digital ID (UAE Pass)	Citizen/business identity	Very High
Payments (Pay.gov)	Government payment rails	Very High
Base Registries	Authoritative data sources	Very High
Cloud Infrastructure	Gov Cloud, Vendor SaaS, Hybrid	High
Security Foundations	IAM, PKI, Network Security	Very High

Platform Team Model

Team	Owns	Consumers
Integration Platform Team	API Gateway, Event Bus, Data Fabric	All capability teams
Developer Experience Team	IDP, Pipelines, Developer Portal	All development teams
Infrastructure Platform Team	Cloud, DPI integration, Security	All platform and capability teams

Anti-Patterns

Anti-Pattern	Problem	Resolution
Ticket-Driven Platform	All requests via tickets	Self-service APIs
Snowflake Infrastructure	Custom setup per team	Golden paths
Shadow IT	Teams build own platforms	Improve platform UX
DPI Bypass	Teams skip shared services	Governance enforcement

Chapter 5

Frames — Cross-Cutting Concerns

Unlike tiers (which stack vertically), frames apply **across** the entire architecture horizontally. In v1.x, Governance and Operations were numbered as "layers" but behaved differently — they govern/observe everything, not just sit above or below.

Frame	Purpose	When Applied	Key Question
Governance	Shape what gets built	Design-Time	Is this allowed? Is this compliant?
Operations	Ensure systems run	Runtime	Is this healthy? Is this secure?

Frame vs. Tier Interaction

The Governance Frame shapes all tiers at design-time through contracts, classifications, and principles. All tiers emit telemetry observed by the Operations Frame at runtime. The Operations Contract bridges both frames.

Aspect	Governance Frame	Operations Frame
When	Design-time	Runtime
Question	Is this allowed?	Is this healthy?
Focus	Compliance, standards	Reliability, performance
Tools	Policies, contracts	Metrics, alerts
Response	Approval/rejection	Scaling, remediation

Chapter 6

Governance Frame

The Governance Frame defines **design-time policies that shape what we build**. It wraps around all tiers horizontally, applying governance at every level.

Core Principles

- 1. Governance Enables, Not Blocks** — "Here's how to do that safely" instead of "You can't do that." Self-service with guardrails, not approval queues.
- 2. Contracts Provide Clarity** — Standard contracts define what "good" looks like, enabling independent evolution.
- 3. Gates Provide Checkpoints, Not Barriers** — Lifecycle gateways ensure quality without bottlenecks.

Governance Bodies

Body	Role	Scope
ARB	Strategic authority	Principle exceptions, capability portfolio, standard contract versions
DDA	Solution governance	Solution reviews, pattern approval, PBC lifecycle
Domain Councils	Domain-specific governance	Domain standards, ownership, ubiquitous language

Standard Contracts

Contract	Purpose	Owner
Security Contract	Security requirements per PBC	Security Team
Data Contract	Data handling, quality, lineage	Data Office
Integration Contract	API/Event standards, schema evolution	Integration Team
NFR Contract	Non-functional requirements	Platform Team
Operations Contract	Runtime requirements, SLOs	Operations Team

Lifecycle Gateways

Gateway	When	Focus
Initiation	Project start	Classification, ownership, alignment, catalog check
Design	Before build	Architecture, contracts, security, principles
Go-Live	Before production	Readiness, operations, compliance, testing
Retirement	End of life	Migration, data, consumer notification

Governance Workflow

- **1. Classify** → Determine criticality, sourcing, stability, PBC type
- **2. Contract** → Identify which contracts apply and version requirements
- **3. Validate** → Automated checks, gateway review if needed
- **4. Operate** → Monitor compliance, report drift, remediate

Anti-Patterns

Anti-Pattern	Problem	Resolution
Governance Theatre	Process without value	Automate and streamline
Exception Inflation	Too many exceptions granted	Improve standards
Stale Standards	Outdated requirements	Regular review cycles
Shadow Governance	Unofficial processes	Consolidate and publish

Chapter 7

Operations Frame

The Operations Frame defines **runtime policies that ensure systems run reliably**. It wraps around all tiers, observing and responding to conditions at every level.

Core Principles

- 1. Observe All Tiers Uniformly** — Every tier emits telemetry; Operations consumes it consistently.
- 2. Proactive Over Reactive** — Detect problems before users do through anomaly detection.
- 3. Automate Response Where Possible** — Playbook-driven response for known scenarios.

Components

Component	Purpose	Key Elements
Security Operations	24/7 security monitoring	SOC, SIEM, Incident Response, Threat Intel
Secrets Management	Centralized secret handling	Key Vault, Rotation, Access Control, Audit
Observability	Full-stack visibility	Metrics, Logs, Traces, Alerting
Capacity Management	Right-sizing resources	Sizing, Auto-scaling, Performance, Cost Optimization
Business Continuity	Disaster recovery & backup	DR Planning, Backup, RTO/RPO, Failover

Recovery Tiers by Criticality

Criticality	RTO	Strategy	Replication
Critical	1 hour	Active-Active	Synchronous
High	4 hours	Active-Passive	Asynchronous
Medium	24 hours	Cold standby	Point-in-time recovery
Low	72 hours	Backup restore	Manual recovery

Operations Workflow

- 1. Observe** → Collect, correlate, baseline, detect anomalies
- 2. Alert** → Threshold breach, notify, acknowledge, triage
- 3. Respond** → Select playbook, execute, validate, communicate
- 4. Improve** → Post-incident review, update playbooks, prevent recurrence

Operations Contract

Aspect	Requirement
SLO	Service Level Objectives per criticality tier
Incident Response	Response times and escalation paths
Backup	Retention periods and recovery testing
Monitoring	Required metrics, logs, traces per PBC
Runbooks	Documented operational procedures

Anti-Patterns

Anti-Pattern	Problem	Resolution
Alert Fatigue	Too many alerts	Tune thresholds, prioritize
Monitoring Gaps	Blind spots	Full-stack observability
Manual Runbooks	Slow response	Automate where possible
DR Theater	Untested plans	Regular DR drills

Chapter 8

Architecture Principles

All PBCs must adhere to 17 architecture principles organized into three categories.

Core Principles (ARC-001 to ARC-010)

ID	Principle	Description
ARC-001	API-First	Expose APIs before UIs; APIs are the primary interface
ARC-002	Cloud-Native First	Design for cloud from the start; containerized, scalable
ARC-003	Security by Design	Embed security from inception, not as afterthought
ARC-004	Privacy by Design	Build in data protection; minimize collection
ARC-005	Zero Trust	Never trust, always verify; least privilege
ARC-006	Accessibility First	WCAG 2.1 AA compliance for all public experiences
ARC-007	Open Standards	Prefer open over proprietary; reduce lock-in
ARC-008	Reuse Before Build	Check the PBC Catalog first; avoid duplication
ARC-009	Loosely Coupled	Minimize dependencies; contract-based integration
ARC-010	Design for Failure	Assume failure will occur; design accordingly

Evolvability Principles — NS-Aligned (ARC-011 to ARC-013)

These principles ensure bounded changes produce bounded work, aligned with Normalized Systems theory.

ID	Principle	Description
ARC-011	Interface Stability	Separate contracts from implementation; contracts change slowly
ARC-012	Tolerant Evolution	Accept unknown fields; be liberal in what you accept
ARC-013	Versioned Compliance	Declare which contract versions a PBC implements

Operational Excellence Principles (ARC-014 to ARC-017)

ID	Principle	Description
ARC-014	Citizen Experience First	Design from the citizen's perspective; measure satisfaction
ARC-015	Data Quality First	Treat data as a product; measure and improve quality

ID	Principle	Description
ARC-016	Observable by Default	Every PBC must emit metrics, logs, and traces
ARC-017	Sustainable by Design	Consider environmental impact; optimize resource usage

Gateway Enforcement

Gateway	Key Principles Validated
Initiation	Cloud-Native, Reuse Before Build, Versioned Compliance
Design	API-First, Security by Design, Interface Stability, Tolerant Evolution
Go-Live	Zero Trust, Accessibility, Design for Failure, Loosely Coupled, Observable

Exception Authority

Principle	Exception Authority
Cloud-Native First	ARB approval required
Open Standards	DDA approval required
Accessibility First	DDA approval (with remediation plan)
Security exceptions	CISO + DDA approval
All others	DDA case-by-case

Chapter 9

Classifications

PBCs are classified across multiple orthogonal dimensions. Classifications are independent — a PBC is classified on each dimension separately.

Dimensions Overview

Dimension	Options	Purpose
Criticality	Critical, High, Medium, Low	Impact assessment and governance level
Sourcing	Build, Buy, Outsource	Delivery strategy
Stability Tiers	Foundational, Stable, Evolving	Change management and interface contracts
PBC Maturity	L0 Concept → L1 Pilot → L2 Production → L3 Optimized	Capability assessment
PBC Type	Application, Analytics, Data, Orchestration, Digital Service, Digital Transaction	Functional classification

Criticality Classification

Level	Description	Governance	Examples
Critical	Government-wide impact	ARB oversight; highest scrutiny	Digital ID, Payments, Citizen Registry
High	Significant entity impact	DDA priority review	Finance, HR, Procurement
Medium	Localized impact	Standard DDA review	Case Mgmt, Email, Collaboration
Low	Minimal impact	Streamlined review	Legacy Integration, Custom Analytics

Determining Criticality

Factor	Critical	High	Medium	Low
Scope of Impact	Government-wide	Multiple entities	Single entity	Team/function
Operational Dependence	Core ops stop	Major disruption	Moderate disruption	Minimal disruption
Data Sensitivity	Citizen PII, national sec	Sensitive internal	Internal	Public
Strategic Importance	Differentiating	Core but commodity	Supporting	Niche

Sourcing Strategy

Strategy	When to Use	Governance
Build	Differentiating capability; no market solution	Full DDA review; internal team ownership
Buy	Commodity capability; strong market solutions	DDA Technical Committee vendor evaluation
Outsource	Specialized capability; skills gap	DDA review with vendor management

Stability Tiers

Tier	Change Frequency	Contract Requirements	Examples
Foundational	Yearly or less	Strictest; multi-year deprecation notice	Digital ID, Payment Rails
Stable	Quarterly	Standard; 6-month deprecation notice	Core PBCs, integration APIs
Evolving	Monthly/Weekly	Flexible; 3-month deprecation notice	UI components, analytics models

PBC Maturity Levels

Level	Name	Description	Characteristics
L0	Concept	Identified need, not yet built	Business case, no code
L1	Pilot	Under development or in pilot	Limited users, iterating
L2	Production	Live and operational	Full governance, contracts met
L3	Optimized	Mature, continuously improved	Metrics-driven optimization

Artefact Lifecycle

Framework artefacts (reference architectures, extensions, catalogs) follow their own lifecycle: **Draft** → **Review** → **Published** → **Deprecated** → **Retired**. This is separate from PBC maturity and governed independently.